



The /book Skill - User Guide

Write your book in your own voice - an AI writing studio for people who aren't AI people

What is /book?

/book is a book-writing studio that runs inside Claude. You talk to it in plain language - no commands to memorize, no prompts to engineer - and a team of six specialized AI agents researches, drafts, edits, and reviews your non-fiction book, chapter by chapter. The finished product is a real Word document with a title page, table of contents, glossary, and endnotes.

What makes it different from asking a chatbot to "write my book": it writes in **your** voice. You feed it anything you have ever written - a handful of tweets, old blog posts, a whole manuscript - and it builds a measurable fingerprint of how you write: how long your sentences run, which words you love, which you never use, how you open and close an argument. Every page it produces is held to that fingerprint, and an editor agent strips out the tell-tale patterns that make AI writing sound like AI.

Your voice, replicated A quantified style fingerprint built from your own writing - strictly enforced	Six specialists Researcher, writer, editor, a simulated reader, a librarian, and a compiler	A real book file Compiled .docx with TOC, glossary and endnotes - ready for an editor or publisher
---	---	--

What you need

- **Claude Code** on any plan - the \$20/month tier works (see "What a session costs" below). Prefer OpenAI? It also runs on Codex CLI and as a ChatGPT GPT - see "Works with OpenAI too" below.
- **Python 3** installed (used only to produce the Word file - no extra packages needed).
- **The skill** - one command installs it:

```
# Windows (PowerShell)

git clone https://github.com/luquiluke/claude-book-skill "$env:USERPROFILE\.claude\skills\book"

# macOS / Linux

git clone https://github.com/luquiluke/claude-book-skill "$HOME/.claude/skills/book"
```

Your first session

Open Claude Code and talk to it the way you would talk to a collaborator:

```
You: I want to write a book about urban beekeeping
You: here are my old blog posts - learn my style
You: write chapter 1
```

That is genuinely all there is to it. Behind those three lines: a short interview turns your idea into a premise, a concrete reader, and a chapter map; the Librarian agent studies your writing samples and runs a quick calibration test ("which of these three paragraphs sounds most like you?"); and the chapter pipeline takes it from there, pausing to ask for your approval at the two moments that matter.

Teaching it your voice

Give it whatever you have - more is better, but even a little works:

- **Files** - drop Word docs, PDFs, or text files into your book's `sources/style/` folder
- **Links** - paste URLs to your blog posts, Substack, or published articles
- **Google Drive or Notion** - if connected to your Claude, it pulls your docs directly
- **Your X/Twitter archive** - request it from X Settings; the skill knows how to read the export
- **Nothing yet?** It interviews you instead and refines the voice as you approve chapters

The fingerprint is strict on purpose. If you genuinely use a word that generic "AI-detector" lists ban - say you really do write *robust* - the skill whitelists it, with evidence from your own texts. Your habits outrank the rules. And it keeps learning: every time you hand-edit a chapter or say "too stiff," the profile updates, with a dated changelog you can read.

TIP: The single best sample you can give it is your own edit of chapter 1. Approve a chapter, mark it up, and tell the skill to learn from your changes.

The team working for you

Agent	What it does for you
Librarian	Studies your writing, builds the voice fingerprint, keeps the book's memory (terms, examples, promises made to the reader)
Researcher	Reads your source material first, searches the web only to fill gaps - every fact carries a source, or a visible "needs verification" flag
Writer	Drafts the full chapter in your voice, following your book's chapter template
Editor	Line-edits and removes AI tells - calibrated to your fingerprint, so it never sands your voice off
Reviewer	Pretends to be your target reader and flags anything that reader would stumble on
Compiler	Assembles the finished Word file: title page, TOC, chapters, glossary, endnotes

Writing a chapter, step by step

Step	What happens	You
1	Researcher builds a fact sheet for the chapter	
2	Checkpoint - you review the facts: anything wrong or missing?	Approve
3	Writer drafts the chapter in your voice	
4	Checkpoint - you read the draft: is it going the right way?	Approve
5	Editor polishes and strips AI patterns	
6	Reviewer reads it as your reader; unclear passages go back to the Editor (up to 3 rounds)	
7	Chapter saved; the book's memory and your voice profile update	

Guided mode (default) Pauses at both checkpoints. Best quality control - errors get caught before they reach the prose.	Express mode (say "express") Runs the whole pipeline and shows you only the finished chapter. Faster and cheaper.
---	---

Say it - don't command it

There is no syntax to learn. These all just work:

You say...	The studio...
"I want to write a book about X"	Starts the setup interview
"Here's my writing - learn my style"	Ingests samples and builds your fingerprint
"Write the next chapter"	Runs the full pipeline on the next chapter
"That doesn't sound like me"	Recalibrates the voice profile
"I already have drafts"	Imports them - keeping your structure, or mining them as raw material (your choice)
"Tighten up chapter 3"	Editor-only polish pass
"Give me the Word file"	Compiles the book to .docx
"How's the book coming?"	Shows a chapter-by-chapter status table

Prefer shortcuts? They exist: `/book new · ingest · style · outline · write N · import · polish N · review N · compile · status · list`

Your book is a folder you own

Everything lives in plain files you can open, read, and edit - no lock-in, nothing hidden:

File / folder	What it is
book.md	Your premise, reader, chapter template, and chapter map
style-profile.md	Your voice fingerprint - with a changelog of everything it has learned
state.md	The book's memory: terminology, recurring examples, facts, promises to the reader
sources/	Where you drop writing samples and research material
research/	Per-chapter fact sheets, every claim sourced
manuscript/	The chapters themselves
versions/	A timestamped backup of every file before every change - nothing is ever lost
output/	The compiled Word document

The finished book

"Give me the Word file" produces a complete **.docx**: title page, table of contents, every chapter starting on a fresh page, a glossary built automatically from the terms your book defined, endnotes drawn from the fact sheets, plus a dedication and about-the-author page if you want them.

Want it to match a specific look? Drop any Word document into `sources/sample.docx` and the Compiler clones its fonts, styles, and page setup. Otherwise you get a clean, professional book template.

TIP: When you first open the compiled file, Word will ask to update the table of contents. Right-click it and choose "Update Field" - that is expected, not an error.

What a session costs

GOOD TO KNOW: A full guided chapter runs 5-8 AI agents (research, draft, edit, up to three review rounds). On a \$20/month plan that is a meaningful share of a session window - plan on one or two chapters per sitting. Express mode costs less; working one stage at a time costs least. The skill tells you this up front and never blocks you.

Works with OpenAI too

The skill format is an open standard, so the same studio runs outside Claude. On **OpenAI Codex CLI** (and Gemini CLI, Copilot, Cursor), two commands install it, and the agents run as isolated child processes:

```
git clone https://github.com/luquilluke/claude-book-skill "$HOME/.codex/skills/book"
cp "$HOME/.codex/skills/book/ports/codex/SKILL.md" "$HOME/.codex/skills/book/SKILL.md"
```

On **ChatGPT**, Book Studio becomes a Custom GPT you build in about ten minutes from the repo's `ports/chatgpt/SETUP.md`: the full six-role pipeline with both checkpoints, your book traveling as a single book.zip between sessions, and the same Word compiler running inside Code Interpreter. ChatGPT Plus is enough.

En español - voseo incluido

El estudio escribe en cualquier idioma, y el español rioplatense es ciudadano de primera clase. Si tu corpus vosea, el libro vosea - *tenés, querés, hacé, andá* - sin deslizarse jamás al tuteo ni al español peninsular. El Editor verifica cada verbo en segunda persona, caza las muletillas típicas de la IA en español ("cabe destacar", "no solo... sino también") y los calcos del inglés ("hacer sentido", "tomar acción"). El nivel de lunfardo lo decide tu propio corpus: la gramática no se negocia, el color es todo tuyo.

Quick answers

Do I need to know anything about AI?

No. If you can describe your book in a sentence, you can use this.

How much writing do I need to provide?

Anything works. A few tweets give it a rough sketch; a book's worth gives it near-perfect pitch. It tells you honestly how confident the profile is.

Will it make things up?

Facts come from your sources or from research with a cited link. Anything it could not verify is flagged for you to confirm - it is never silently invented.

What if I hate a chapter?

Say so. Every prior version is kept in the versions/ folder, and your criticism teaches the voice profile what to do differently.

Can I use my existing half-written manuscript?

Yes - "I already have drafts." You choose per chapter whether it preserves your structure or mines the draft as raw material.

Is my writing shared anywhere?

Your book folder lives on your own computer. Nothing is uploaded except the conversations you have with Claude itself.

Slater Consulting | The /book skill is open source (MIT): github.com/luquilluke/claude-book-skill | Questions or ideas: open an issue on GitHub